

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ ФАКУЛЬТЕТ

ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1 по курсу «ООП» Тема: Система управления
курсами и преподавателями.

Вариант 25 Выполнил: Чекушин В.С. К3241

Преподаватель: Николай Кочубеев

Санкт-Петербург 2025 г.

Оглавление

Оглавление:Ошибка! Закладка не определена.

Структура проекта 3

Базовые сущности (Student, Teacher) 3

Реализация наследования (Online/Offline Courses)..... 3

Логика управления курсами 4

Unit-тестирование 4

Вывод 4

Структура проекта

Программа построена на принципах ООП (наследование, инкапсуляция, полиморфизм) и разделена на логические блоки:

- Интерфейсы: задают стандарты для сущностей системы.
- Модели данных: классы Student и Teacher, инкапсулирующие информацию об участниках обучения.
- Иерархия курсов: базовый абстрактный класс Course и его наследники для разных форм обучения.
- Управление: класс CourseManager для обработки данных и поиска.
- Тестирование: Unit-тесты на базе xUnit для верификации бизнес-логики.

Базовые сущности (Student, Teacher)

Простые классы-контейнеры для хранения имен пользователей.

```
public class Teacher
{
    public string Name { get; set; }
    public Teacher(string name) { Name = name; }
}
```

Реализация наследования (Online/Offline Courses)

Абстрактный класс Course содержит общую логику добавления студентов, а наследники расширяют её специфическими полями (платформа или аудитория).

```
public class OnlineCourse : Course
{
    public string Platform { get; set; }

    public OnlineCourse(string name, Teacher teacher, string platform)
        : base(name, teacher)
    {
        Platform = platform;
    }
}

public class OfflineCourse : Course
{
    public string Room { get; set; }

    public OfflineCourse(string name, Teacher teacher, string room)
        : base(name, teacher)
    {
        Room = room;
    }
}
```

```
{  
    Room = room;  
}
```

Логика управления курсами

Класс CourseManager реализует методы для работы с коллекцией всех курсов системы.

```
public List<Course> GetCoursesByTeacher(Teacher t)  
{  
    List<Course> result = new List<Course>();  
    foreach (var c in AllCourses)  
    {  
        if (c.Teacher.Name == t.Name)  
            result.Add(c);  
    }  
    return result;  
}
```

Unit-тестирование

Для проверки корректности работы системы реализованы тесты, проверяющие добавление курсов, регистрацию студентов и фильтрацию по преподавателю.

```
[Fact]  
public void GetCoursesByTeacher_ReturnsCorrectCourses()  
{  
    CourseManager manager = new CourseManager();  
    Teacher t1 = new Teacher("Иван");  
    var c1 = new OnlineCourse("C#", t1, "Teams");  
  
    manager.AddCourse(c1);  
    var result = manager.GetCoursesByTeacher(t1);  
  
    Assert.Equal(1, result.Count);  
    Assert.Equal("C#", result[0].Name);  
}
```

Вывод

- **Полиморфизм:** использование базового типа Course в списке AllCourses позволяет работать с разными типами курсов единообразно.
- **Разделение ответственности:** логика хранения данных (CourseManager) отделена от интерфейса пользователя (Program).
- **Тестируемость:** архитектура позволяет легко проверять методы через автоматизированные тесты без запуска консольного интерфейса.